Information Retrieval

UNIT 1

Introduction

Introduction to information retrieval, Major challenges in IR, Features of an IR system, components of an IR model, IR system block diagram, Boolean retrieval, Information versus Data Retrieval, Text categorization, IR processes and fields, Vector Model, Probabilistic Model and Latent Semantic Indexing Model.

1) Write short note on Vector Model & Probabilistic Model.

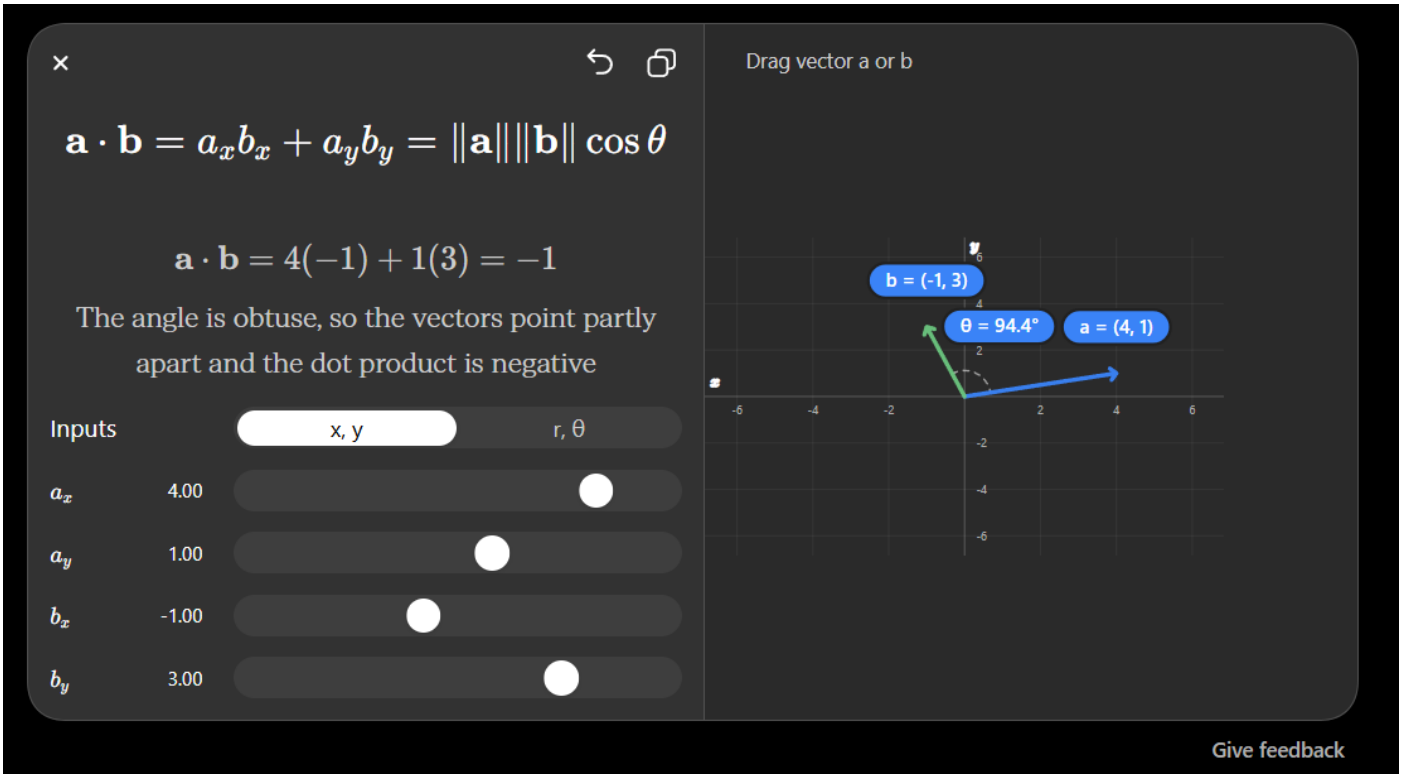
1. Vector Space Model

The **Vector Space Model (VSM)** represents documents and user queries as vectors in a multidimensional space. Each dimension represents a term, and the value represents the importance/weight of that term.

For example, if the terms are `{data, mining, learning}`, a document can be represented as:

$$D = (0.5, 0.8, 0.2)$$

The similarity between a query and document is commonly calculated using **Cosine Similarity**:



$$Similarity(D, Q) = \frac{D \cdot Q}{|D||Q|}$$

A higher cosine similarity indicates that the document is more relevant to the query.

Advantages:

- Supports ranked retrieval.
- Allows partial matching.
- Can use TF-IDF for term weighting.
- Simple mathematical representation.

Disadvantages:

- Produces high-dimensional vectors.
- Assumes terms are independent.
- Does not directly understand the semantic meaning of words.

2. Probabilistic Model

The **Probabilistic Information Retrieval Model** ranks documents according to the probability that they are relevant to a given query.

The basic idea is to estimate:

$$P(R = 1|D, Q)$$

where:

- $R = 1$ means the document is relevant.
- D is the document.
- Q is the query.

The general process is:

```
User Query
    ↓
Estimate Relevance Probability
    ↓
Calculate Document Scores
    ↓
Rank Documents
    ↓
Display Relevant Documents
```

Documents having a **higher probability of relevance** are placed higher in the search results.

Advantages:

- Provides ranked search results.
- Based on a strong mathematical foundation.
- Can use relevance feedback to improve results.
- Focuses directly on document relevance.

Disadvantages:

- Initial relevance information may not be available.
- Estimating probabilities can be difficult.
- The model may require assumptions about terms and relevance.

Difference

Vector Model	Probabilistic Model
Represents documents as vectors	Represents relevance using probabilities
Uses similarity measures	Uses probability of relevance
Commonly uses cosine similarity	Uses relevance probability
TF-IDF can be used for weighting	Relevance estimation is central
Supports ranked retrieval	Supports ranked retrieval

2) Enlist features of IR system

Features of an Information Retrieval (IR) System

An **Information Retrieval (IR) system** is designed to find and retrieve relevant information from a large collection of documents.

Important Features:

1. **Efficient Searching**
   Searches a large number of documents quickly using indexes.
2. **Relevance of Results**
   Retrieves documents that are relevant to the user's query.
3. **Ranking of Results**
   Ranks retrieved documents according to their relevance or similarity to the query.
4. **Query Processing**
   Processes user queries by techniques such as tokenization, stop-word removal and stemming.
5. **Indexing**
   Creates an index of terms to make document retrieval faster.
6. **Large-Scale Handling**
   Can efficiently handle large collections of documents.
7. **Support for Different Queries**
   Supports Boolean, keyword, phrase and ranked queries.
8. **Fast Response Time**
   Provides search results within a short time.
9. **Updating Capability**
   Allows documents to be added, modified or deleted from the collection.
10. **User-Friendly Interface**
    Presents search results in a simple and understandable form.

In short:

The main features of an IR system are:
fast searching, indexing, query processing, relevance, ranking, scalability, updating and user-friendly result presentation.

3) Explain methods of text categorization.

Methods of Text Categorization

Text categorization is the process of automatically assigning a text document to one or more predefined categories based on its content.

Example:



Main Methods of Text Categorization

1. Rule-Based Method

In this method, predefined **rules and keywords** are used to assign documents to categories.

Example:



Advantages:

- Simple to understand.
- Easy to implement for small datasets.
- No training data is required.

Disadvantages:

- Rules must be created manually.
- Difficult to maintain for large datasets.
- Cannot easily handle new topics or language variations.

2. Naive Bayes Classification

Naive Bayes is a **probabilistic machine-learning method** based on Bayes' theorem.

It calculates the probability that a document belongs to each category and selects the category with the highest probability.

For a document D and category C:

$$P(C|D) = \frac{P(D|C)P(C)}{P(D)}$$

Example:

```
Document → "team won the cricket match"
                ↓
         Naive Bayes
                ↓
             Sports
```

Advantages:

- Fast and simple.
- Works well with text data.
- Requires relatively small training datasets.

3. Decision Tree Method

A **Decision Tree** classifies documents by making a sequence of decisions based on selected features or terms.

Example:

```
        Contains "goal"?
           /       \
         Yes        No
          ↓          ↓
      Sports    Contains "election"?
                   /       \
                 Yes        No
                  ↓          ↓
              Politics     Other
```

Advantages:

- Easy to understand.
- Easy to visualize.
- Useful for classification based on clear features.

Disadvantages:

- Can become very large.
- May overfit the training data.

4. Support Vector Machine (SVM)

SVM classifies documents by finding an optimal decision boundary that separates different categories.

For example:



SVM is highly effective for **high-dimensional text data**.

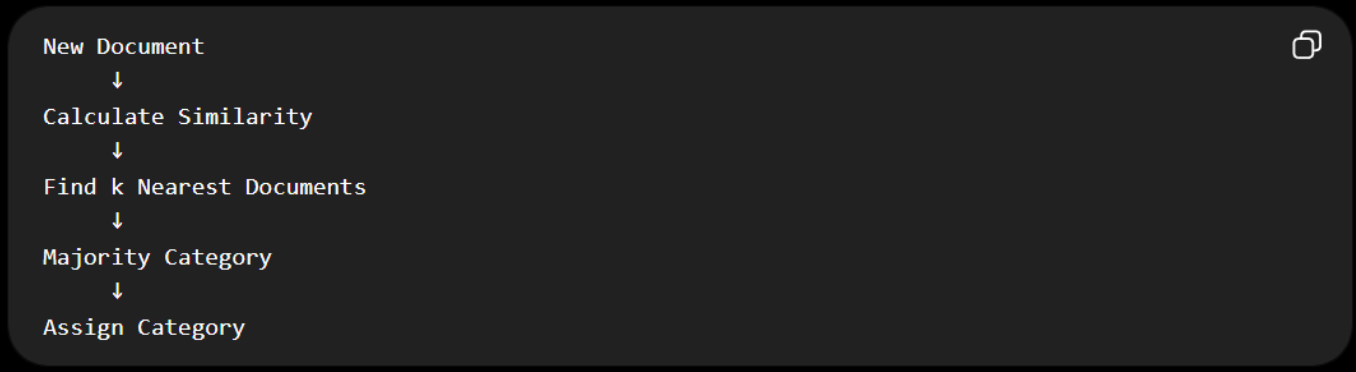
Advantages:

- High classification accuracy.
- Works well with large numbers of text features.
- Effective for sparse data.

5. k-Nearest Neighbour (k-NN)

In **k-NN**, a new document is classified according to the categories of its most similar documents.

Process:



Example:
If 4 out of the 5 nearest documents belong to *Sports*, the new document is classified as **Sports**.

Advantages:

- Simple to implement.
- No complex training phase.
- Effective for smaller datasets.

6. Neural Network / Deep Learning Method

Neural networks learn complex relationships between words and documents from training data.

Modern systems may use:

- Neural Networks
- CNN
- RNN
- Transformers

They can classify documents based on their semantic and contextual information.

Example:

```
Customer Review
      ↓
Neural Network
      ↓
Positive / Negative
```

Conclusion

The major methods of text categorization are **Rule-Based, Naive Bayes, Decision Tree, SVM, k-NN and Neural Network methods**. Among these, **Naive Bayes and SVM** are traditionally very important for text classification because they work effectively with high-dimensional textual data.

4) What are the Components of IR model?

Components of an Information Retrieval (IR) Model

An **Information Retrieval (IR) model** defines how documents and queries are represented and how the relevance of a document to a query is determined.

The main components are:

1. Document Representation

Documents are converted into a suitable representation using terms or features.

Example:

```
Document: "Machine learning uses data"
Terms: {machine, learning, uses, data}
```

2. Query Representation

The user's query is converted into a form that can be compared with document representations.

Example:

```
Query: "machine learning"
Terms: {machine, learning}
```

3. Matching Function

The matching function determines how closely a document matches the user's query.

For example:

- Boolean matching
- Cosine similarity
- Probability of relevance

4. Relevance Measure

It determines how relevant a document is to a particular query.

For ranked retrieval, a **relevance score** is generally calculated for each document.

5. Ranking Function

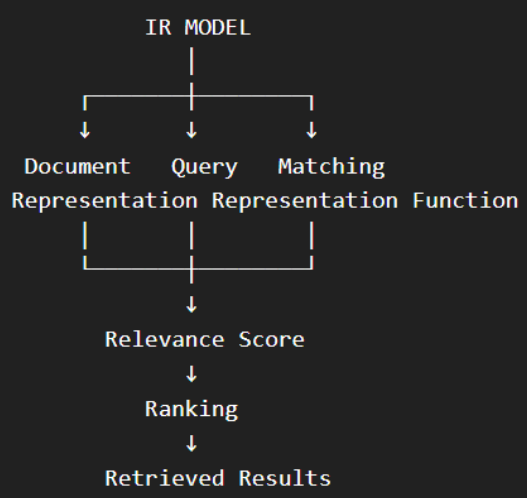
Documents are arranged according to their relevance scores, with the most relevant documents appearing first.

```
Query
  ↓
Matching
  ↓
Relevance Score
  ↓
Ranking
  ↓
D1 → D3 → D5 → D2
```

6. Retrieval Mechanism

The system retrieves documents from the collection or index that satisfy the query.

Basic Structure



In Short

The important components of an IR model are:

Document Representation → Query Representation → Matching Function → Relevance Measure → Ranking Function → Retrieval Mechanism.

5) Differentiate between Data retrieval & Information retrieval.

Data Retrieval	Information Retrieval
1. Retrieves **exact data** from a database.	1. Retrieves **relevant information** from a collection of documents.
2. Mainly deals with **structured data**.	2. Mainly deals with **unstructured or semi-structured data**.
3. Uses structured query languages such as **SQL**.	3. Uses **natural language, keywords, Boolean and ranked queries**.
4. Results are generally **exact and deterministic**.	4. Results are generally **ranked according to relevance**.
5. Exact matching is usually required.	5. **Partial and approximate matching** is possible.
6. Does not usually consider the relevance of results.	6. **Relevance of retrieved documents** is very important.
7. Usually works with a **database schema**.	7. Usually works with a **document collection and index**.
8. Example: Finding a student with **Roll No. 25**.	8. Example: Searching **"machine learning applications"** on Google.
9. Commonly uses **database management systems (DBMS)**.	9. Commonly uses **search engines and IR systems**.
10. Suitable for retrieving **specific records or values**.	10. Suitable for finding **relevant documents or information**.

Key Difference

Data Retrieval → Exact data

Information Retrieval → Relevant information

6) What is Boolean Retrieval?

Definition

Boolean Retrieval is an Information Retrieval technique in which documents are retrieved by matching the user's query using **Boolean operators** such as **AND, OR, and NOT**.

A document either **satisfies the query or does not satisfy it**.

Boolean Operators

Operator	Meaning	Example
AND	Both terms must be present	`Machine AND Learning`
OR	At least one term must be present	`Machine OR Learning`
NOT	Excludes a term	`Machine NOT Learning`

Example

Suppose we have:

```
D1 = Machine Learning
D2 = Machine Intelligence
D3 = Deep Learning
```

Query:

```
Machine AND Learning
```

Result:

```
D1
```

because D1 contains both **Machine** and **Learning**.

Working

```
User Query
    ↓
Boolean Operators
    ↓
Search Inverted Index
    ↓
Match Documents
    ↓
Retrieved Documents
```

Advantages

- Simple and easy to understand.
- Provides precise control over searching.
- Efficient for exact matching.
- Easy to implement using an inverted index.

Limitations

- Does not rank the retrieved documents.
- May return too many or too few documents.
- Users need to construct appropriate Boolean queries.

In short: Boolean Retrieval uses **AND, OR and NOT** to determine whether a document matches a query.

7) Explain Major Challenges in IR.

Information Retrieval (IR) aims to find relevant information from a large collection of documents. However, several challenges make efficient and accurate retrieval difficult.

1. Large Volume of Data

Modern systems contain millions or billions of documents. Processing such a huge collection efficiently requires effective **indexing and searching techniques**.

2. Ambiguity of Language

The same word can have different meanings depending on context.



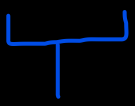
Example:

`Bank` → financial institution or river bank.

This makes it difficult to determine the user's actual intention.

3. Synonymy

Different words can represent the same meaning.



Example:

```
Car ⇔ Automobile ⇔ Vehicle
```

A document containing "automobile" may be relevant to a query containing "car", even though the exact word is absent.

4. Polysemy

A single word may have multiple meanings.

Example:

`Apple` → fruit or technology company.

The IR system must identify the correct meaning from context.

5. Ranking Relevant Documents

A query may retrieve thousands of documents. The system must determine which documents are **most relevant** and place them at the top of the results.

6. Query Ambiguity

Users often submit short or unclear queries.

Example:

```
"Python"
```

This could refer to the programming language or the snake.

7. Dynamic and Changing Data

Web pages and documents are continuously added, modified and deleted. Therefore, the index must be **updated regularly**.

8. Multilingual Information

Documents may be written in different languages. Handling different languages, scripts, grammar and meanings is a major challenge.

9. Precision and Recall

An IR system should retrieve as many relevant documents as possible while avoiding irrelevant documents.

- **Precision** → How many retrieved documents are relevant?
- **Recall** → How many relevant documents were successfully retrieved?

10. Scalability and Response Time

The system must provide results **quickly**, even when the document collection and number of users become very large.

Conclusion

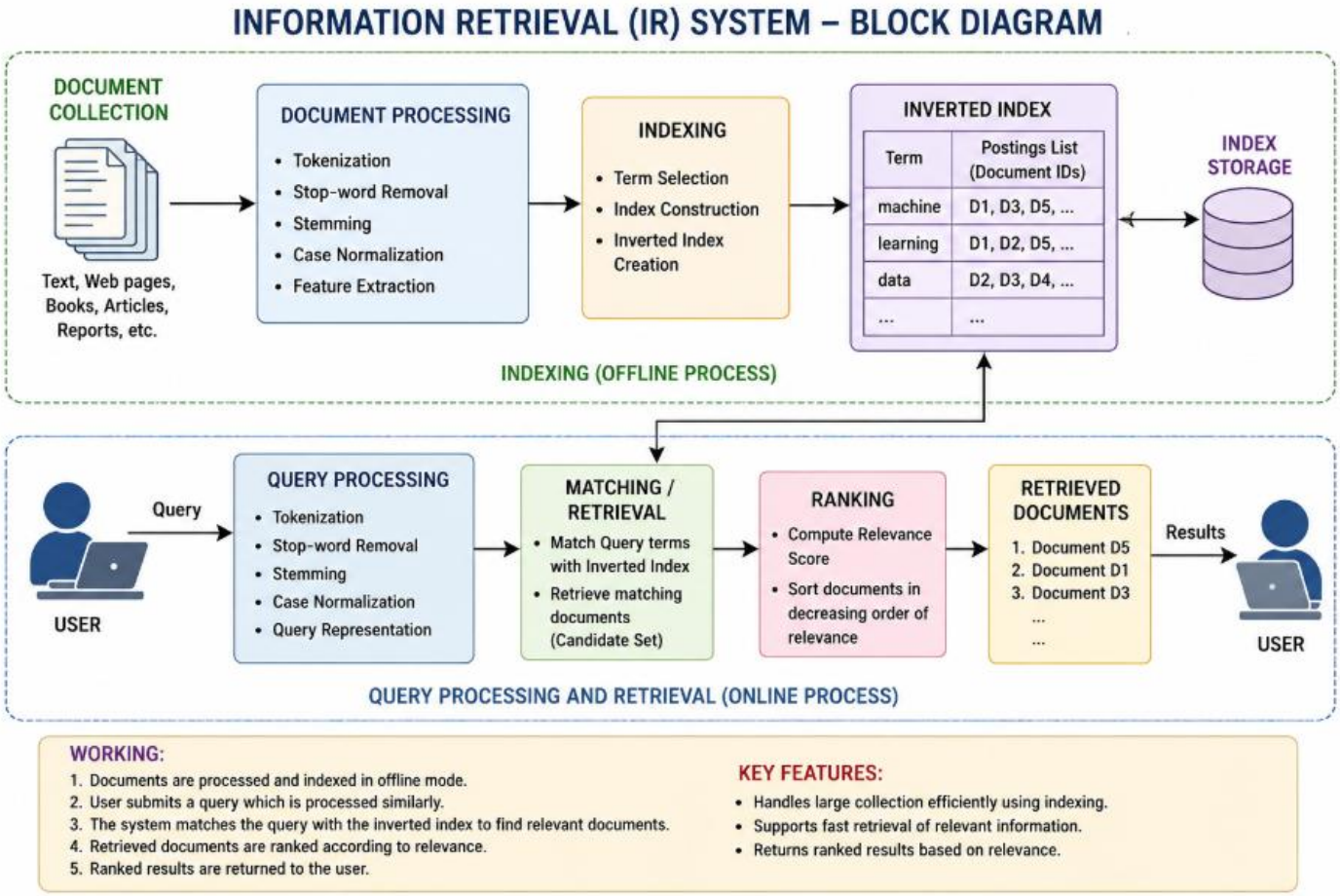
The major challenges of IR are **large data volume, language ambiguity, synonymy, polysemy, query interpretation, ranking, dynamic data, multilingual content, precision-recall trade-off and scalability**. An effective IR system must address these challenges to provide **fast and relevant search results**.

8) Draw and Explain Block Diagram of IR System.

Block Diagram of Information Retrieval (IR) System

An **Information Retrieval (IR) system** is used to retrieve relevant documents from a large collection based on a user's query.

Block Diagram



Explanation of Components

1. Document Collection

It contains a large collection of documents such as **web pages, books, articles, emails and reports**.

2. Document Processing

Documents are processed to extract useful terms. Common operations include:

- Tokenization
- Stop-word removal
- Stemming
- Case normalization

3. Indexing

The processed documents are indexed to enable **fast searching**. An **inverted index** is commonly used.

4. Inverted Index

It stores terms along with the documents in which those terms occur.

Example:

```
machine  → D1, D3, D5
learning → D1, D2, D5
```

5. Query Processing

The user's query is processed in a similar manner to the documents.

Example:

```
"Machine Learning"
       ↓
machine, learning
```

6. Matching / Retrieval

The processed query is matched against the index to find documents containing relevant terms.

7. Ranking

The retrieved documents are assigned relevance scores and arranged from **most relevant to least relevant**.

8. Retrieved Documents

The ranked results are presented to the user.

Conclusion

The basic working of an IR system is:

Document Collection → Processing → Indexing → Query Processing → Matching → Ranking → Results

The main objective is to provide the user with **relevant information quickly and efficiently**.

UNIT 2

Dictionaries and Query Processing

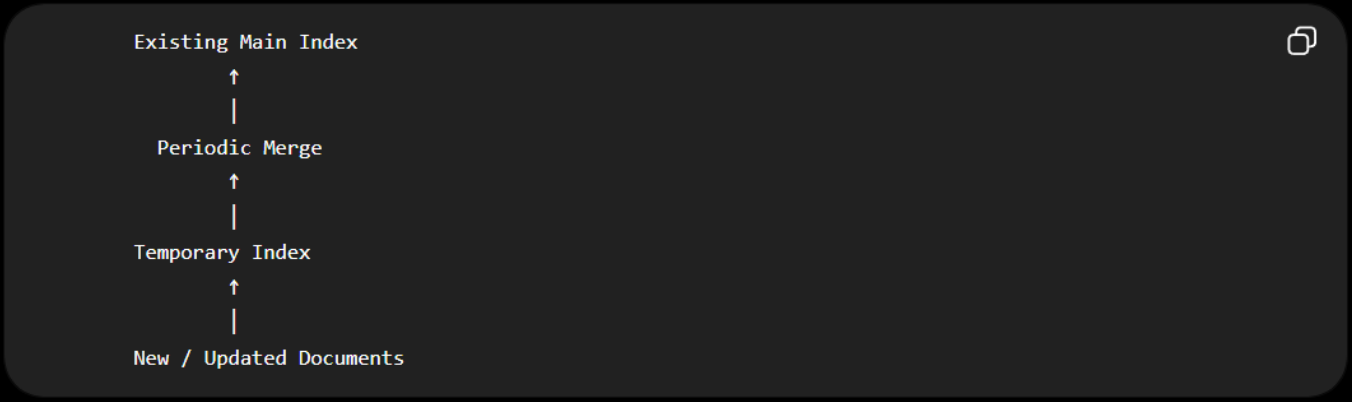
Components of Index, Index Life Cycle, Static Inverted Index, Dictionaries-Types (Sort Based, Hash Based, Interleaving & Posting Lists), Index Construction (In memory, Sort Based, Merge Based, Disk Based Index Construction), Dynamic Indexing, Query Processing for Ranked Retrieval, Document at a Time Query Processing, Term at a Time Query Processing, Pre-computing Score Contributions, Impact Ordering, Query Optimization.

9) Write Short Note on following. i) Dynamic Indexing ii) Impact Ordering.

i) Dynamic Indexing

Dynamic Indexing is a technique used to maintain an inverted index when documents are **frequently added, modified, or deleted**. It avoids rebuilding the entire index whenever new documents arrive.

Working



Explanation

1. The existing documents are stored in the **main index**.
2. New documents are processed and added to a **temporary index**.
3. Queries can search both the main and temporary indexes.
4. Periodically, the temporary index is **merged with the main index**.
5. Deleted or modified documents are handled during index maintenance.

Advantages

- Supports frequent document updates.
- Avoids rebuilding the complete index.
- Suitable for dynamic collections such as web pages.
- Provides updated search results.

Disadvantages

- Multiple indexes may need to be searched.
- Periodic merging requires additional processing.
- More complex than static indexing.

Example: Search engines continuously receive new web pages, so dynamic indexing allows them to update their indexes without rebuilding everything.

ii) Impact Ordering

Impact Ordering is a query-processing technique in which postings are arranged according to their **contribution or impact on the relevance score** of a document.

High-impact postings are processed **before low-impact postings**.

Example

Suppose the score contributions are:

Document	Impact
D1	0.90
D2	0.60
D3	0.20

The impact order is:

```
D1 → D2 → D3
```

Working

```
      Query
        ↓
   Posting Lists
        ↓
 Calculate Impact
        ↓
High Impact First
        ↓
Calculate / Update Scores
        ↓
   Ranked Results
```

Advantages

- High-scoring documents can be identified earlier.
- Reduces unnecessary processing.
- Improves efficiency of ranked retrieval.
- Useful for **Top-k retrieval**.

Conclusion

Dynamic Indexing deals with efficiently **updating the index**, whereas **Impact Ordering** improves the efficiency of **query processing and ranking** by processing high-impact postings first.

10) Explain the static inverted index.

Definition

A **Static Inverted Index** is a data structure used in Information Retrieval that maps each **term (word)** to the list of **documents in which that term occurs**.

It is called **static** because the document collection is assumed to be fixed or changes very infrequently.

Example

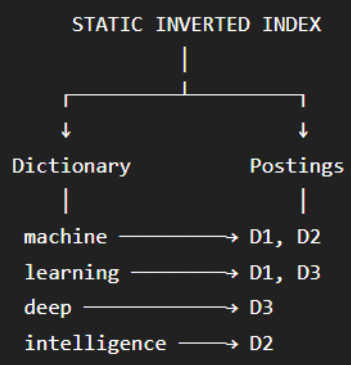
Consider the following documents:

```
D1 → Machine Learning
D2 → Machine Intelligence
D3 → Deep Learning
```

The static inverted index is:

Term	Posting List
deep	D3
intelligence	D2
learning	D1, D3
machine	D1, D2

Structure



Components

1. Dictionary:
Contains all unique terms in the document collection.

2. Posting List:
Contains the document IDs in which each term occurs.

For example:

```
machine → D1, D2
```

means the term **machine** occurs in documents D1 and D2.

Working

1. Documents are collected.
2. Documents are processed using tokenization, stop-word removal and stemming.
3. Unique terms are stored in the dictionary.
4. For every term, its document IDs are stored in a posting list.
5. During searching, the query terms are looked up in the dictionary.
6. The corresponding posting lists are accessed to retrieve documents.

Advantages

- **Fast query processing** because the system directly accesses posting lists.
- Simple and efficient structure.
- Requires less processing during query time.
- Suitable for fixed document collections.
- Supports Boolean and ranked retrieval.

Disadvantages

- Difficult to update frequently.
- Adding or deleting many documents may require index modification or rebuilding.
- Not suitable for highly dynamic collections.

Example Application

A **digital library containing a fixed collection of books** can use a static inverted index because the documents do not change frequently.

Conclusion

A static inverted index provides an efficient mapping:

$$\boxed{\text{Term} \rightarrow \text{List of Documents}}$$

It is highly effective for **fixed document collections**, but dynamic indexing is preferred when documents are frequently added or modified.

11) What are the components of index?

Components of an Index

An **index** in an Information Retrieval (IR) system is a data structure that helps in **fast and efficient retrieval of documents**. The main components of an index are:

1. Dictionary

The **dictionary** contains all the unique terms present in the document collection.

Example:

```
apple
data
learning
machine
network
```

It helps the system quickly locate a particular search term.

2. Postings List

A **postings list** contains the list of documents in which a particular term occurs.

Example:

```
machine  → D1, D3, D5
learning → D1, D2, D5
```

Here, `D1, D3, D5` are the documents containing the term **machine**.

---

3. Document Information

The index may store additional information about each document, such as:

- Document ID
- Document length
- Document location
- Document frequency information

This information is useful during retrieval and ranking.

4. Term Statistics

Term statistics help calculate the importance and relevance of terms.

Important statistics include:

- **Term Frequency (TF):** Number of times a term occurs in a document.
- **Document Frequency (DF):** Number of documents containing the term.
- **Collection Frequency:** Total number of occurrences of a term in the entire collection.

5. Positional Information

For some IR systems, the index stores the **position of a term within a document**.

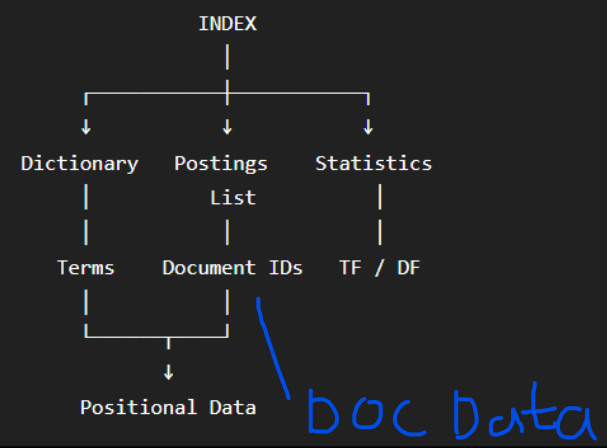
Example:

```
learning → D1: positions 2, 5
```

This is useful for:

- Phrase searching
- Proximity searching
- Advanced ranking

Structure of an Index



In Short

The major components are:

Dictionary + Postings List + Document Information + Term Statistics + Positional Information

Together, these components allow an IR system to **search, retrieve and rank documents efficiently**.

12) Discuss different types of dictionaries in information retrieval.

Types of Dictionaries in Information Retrieval

A **dictionary** in an Information Retrieval (IR) system contains the collection of **unique terms** along with information needed to locate their corresponding postings lists.

The main types of dictionaries are:

1. Sort-Based Dictionary

In a **sort-based dictionary**, all terms are stored in **sorted/alphabetical order**.

Example

```
apple
computer
data
information
machine
network
```

The sorted structure allows efficient searching using **binary search**.

Advantages

- Simple to implement.
- Supports binary search.
- Efficient for exact term lookup.
- Supports sequential processing and prefix-based operations.

Disadvantages

- Inserting a new term can be expensive.
- Maintaining sorted order becomes difficult for frequently changing data.

2. Hash-Based Dictionary

In a **hash-based dictionary**, a hash function maps each term to a specific location or bucket.

Structure

```
      Term
       ↓
  Hash Function
       ↓
   Hash Value
       ↓
     Bucket
```

Example:

```
machine → hash() → Bucket 5
data    → hash() → Bucket 2
network → hash() → Bucket 8
```

Advantages

- Very fast exact-term lookup.
- Average lookup time is approximately **O(1)**.
- Suitable for large dictionaries.

Disadvantages

- Hash collisions may occur.
- Not suitable for prefix searching.
- Terms are not stored in sorted order.

3. Interleaving Dictionary

In **interleaving**, dictionary information and postings information are arranged or accessed in an interleaved manner to improve the efficiency of index access.

A dictionary entry generally contains a term and a pointer to its postings:

```
Term        Pointer
_____________________

machine  →   P1
learning →   P2
data     →   P3

P1 → D1, D2, D5
P2 → D1, D3
P3 → D1, D4
```

The pointer allows the system to quickly locate the corresponding postings list.

Advantages

- Efficient access between terms and postings.
- Reduces unnecessary index access.
- Useful for query processing.

4. Posting Lists

A **posting list** stores the documents in which a particular term occurs. It is closely associated with the dictionary in an inverted index.

Example

```
Dictionary        Posting List

machine      →    D1, D3, D5
learning     →    D1, D2, D5
network      →    D2, D4
```

A posting may also contain additional information such as:

- Document ID
- Term frequency
- Position of the term
- Other scoring information

Uses

Posting lists are used for:

- Boolean retrieval
- Ranked retrieval
- Phrase searching
- Query processing

Comparison

Feature	Sort-Based	Hash-Based	Interleaving	Posting Lists
Organization	Sorted terms	Hash buckets	Term + pointer organization	Document occurrences
Search	Binary search	Hash lookup	Pointer-based access	Sequential/optimized access
Exact lookup	Good	Very fast	Fast	Fast
Prefix search	Good	Poor	Depends on organization	Not primary purpose
Updates	Relatively difficult	Easier	Depends on structure	Can require updates
Main purpose	Store/search terms	Fast term lookup	Efficient dictionary-posting access	Store term occurrences

Conclusion

The important dictionary organizations in IR are **sort-based and hash-based dictionaries**, while **interleaving and posting-list organization** help efficiently connect dictionary terms with the documents in which those terms occur.

13) Explain Index Construction in detail.

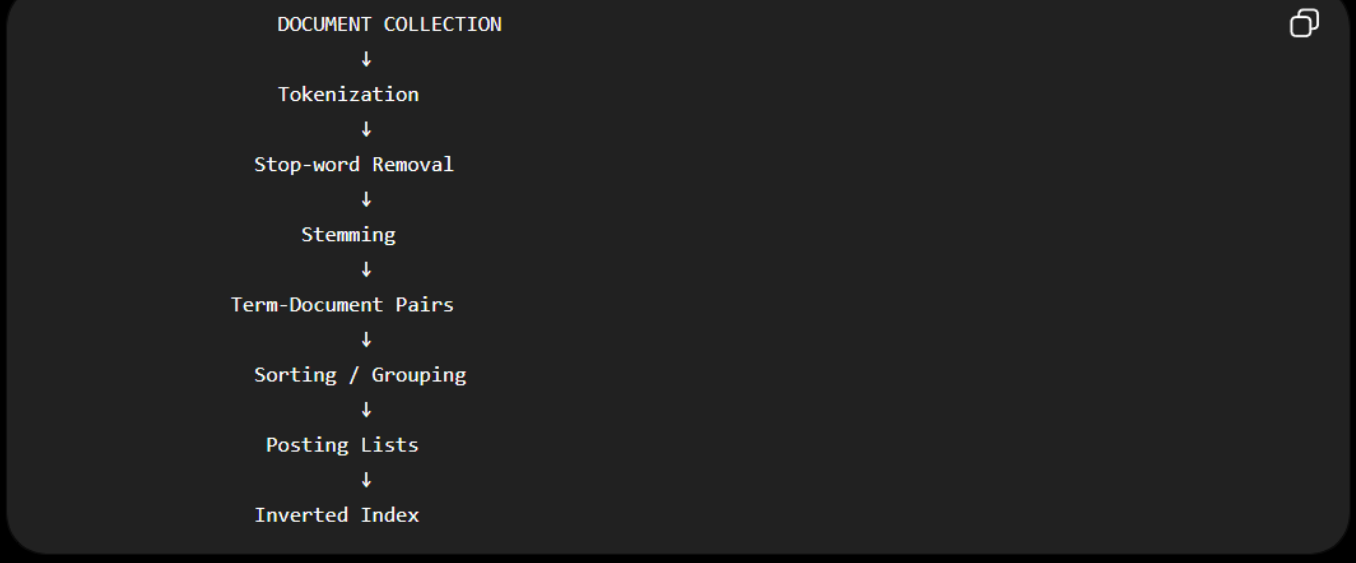
Index Construction in Information Retrieval

Definition

Index Construction is the process of creating an index from a collection of documents so that documents can be **retrieved quickly and efficiently** when a user submits a query.

The most commonly used structure is an **inverted index**, which maps each term to the documents containing that term.

Basic Process



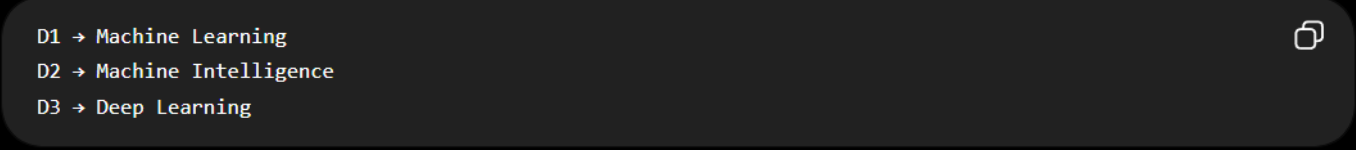
Steps of Index Construction

1. Document Collection

Documents are collected from sources such as:

- Web pages
- Books
- Articles
- Emails
- Reports

Example:



2. Tokenization

The documents are divided into individual terms or **tokens**.

Example:

```
"Machine Learning is useful"

        ↓

Machine | Learning | is | useful
```

3. Stop-Word Removal

Common words that usually provide little information are removed.

Examples:

```
is, the, a, an, of, in, on
```

Therefore:

```
Machine Learning is useful

        ↓

Machine Learning useful
```

4. Stemming / Normalization

Different forms of words can be converted into a common form.

Example:

```
connect
connected
connecting
connection
```

may be reduced to a common stem such as:

```
connect
```

5. Generate Term-Document Pairs

For every term, the corresponding document ID is recorded.

Example:

```
(machine, D1)
(learning, D1)
(machine, D2)
(intelligence, D2)
(deep, D3)
(learning, D3)
```

6. Sort the Terms

The term-document pairs are sorted according to the terms.

```
(deep, D3)
(intelligence, D2)
(learning, D1)
(learning, D3)
(machine, D1)
(machine, D2)
```

7. Create Posting Lists

Documents containing the same term are grouped together.

```
deep         → D3
intelligence → D2
learning     → D1, D3
machine      → D1, D2
```

This forms the **inverted index**.

Advantages of Index Construction

1. Provides **fast document retrieval**.
2. Avoids scanning every document for every query.
3. Supports Boolean and ranked retrieval.
4. Reduces query processing time.
5. Makes large document collections searchable.

Conclusion

Index construction converts a document collection into an efficient **inverted index** consisting mainly of a **dictionary and posting lists**. For small collections, in-memory construction is suitable, while **sort-based, merge-based and disk-based methods** are used for large collections. Thus, index construction is a fundamental step in building an efficient Information Retrieval system.

14) Explain Query Processing for Ranked Retrieval.

Query Processing for Ranked Retrieval

Definition

Ranked Retrieval is an Information Retrieval technique in which documents matching a user's query are **ranked according to their relevance score**. The most relevant documents are displayed first.

Unlike Boolean retrieval, ranked retrieval does not simply return documents as **relevant or non-relevant**. It assigns a score to each document.

Working of Query Processing

The main steps are:

```
USER QUERY
    ↓
Query Processing
    ↓
Query Terms
    ↓
Inverted Index
    ↓
Retrieve Posting Lists
    ↓
Calculate Scores
    ↓
Rank Documents
    ↓
Select Top-k
    ↓
Search Results
```

1. User Query

The user enters a query.

Example:

```
"machine learning"
```

2. Query Processing

The query is processed using techniques such as:

- Tokenization
- Stop-word removal
- Stemming
- Normalization

Example:

```
"Machine Learning"

        ↓

machine, learning
```

3. Access Inverted Index

The system searches the dictionary for the query terms and obtains their posting lists.

Example:

```
machine  → D1, D2, D5
learning → D1, D3, D5
```

---

4. Retrieve Candidate Documents

Documents occurring in the posting lists become **candidate documents**.

```
D1, D2, D3, D5
```

5. Calculate Relevance Score

A score is calculated for every candidate document based on the query and document terms.

A common approach uses **TF-IDF**:

$$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$$

where:

- TF = frequency of term in the document
- IDF = importance of the term across the collection

For vector-space retrieval, **cosine similarity** can be used:

$$Similarity(D,Q) = \frac{D \cdot Q}{|D||Q|}$$

A higher score means greater similarity/relevance.

6. Rank Documents

The documents are sorted according to their relevance scores.

Example:

Document	Relevance Score
D1	0.95
D5	0.82
D3	0.61
D2	0.35

Therefore, the ranking is:

```
D1 → D5 → D3 → D2
```

7. Return Top-k Results

Usually, the system does not need to display every matching document. It returns the **top-k most relevant documents**.

For example, if $k = 3$:

```
1. D1
2. D5
3. D3
```

Techniques Used to Improve Ranked Retrieval

1. Document-at-a-Time (DAAT)

Processes one document at a time and combines contributions from all query terms.

2. Term-at-a-Time (TAAT)

Processes one query term at a time and updates document scores.

3. Impact Ordering

Processes postings having higher score contributions first.

4. Pre-computing Score Contributions

Some scoring values are calculated during indexing to reduce query-time computation.

5. Query Optimization

Techniques such as skip pointers, caching and efficient term ordering reduce processing time.

Advantages

- Produces **ordered and relevant results**.
- Handles partial matches.
- Provides better results than exact Boolean matching for many queries.
- Can return the **top-k** most relevant documents.
- Widely used in modern search engines.

Conclusion

Query processing for ranked retrieval involves **processing the query, accessing posting lists, calculating relevance scores, ranking documents, and returning the top-k results**. The objective is to provide the user with the **most relevant documents first while minimizing query processing time**.

15) What is index lifecycle in information retrieval?

Definition

The **Index Life Cycle** is the sequence of stages through which an index passes, from **creation and storage to updating and maintenance**. It ensures that the IR system can retrieve information **quickly and accurately** from a document collection.

Diagram



Stages of Index Life Cycle

1. Document Collection

Documents are collected from different sources such as web pages, books, articles, emails, etc.

2. Document Processing

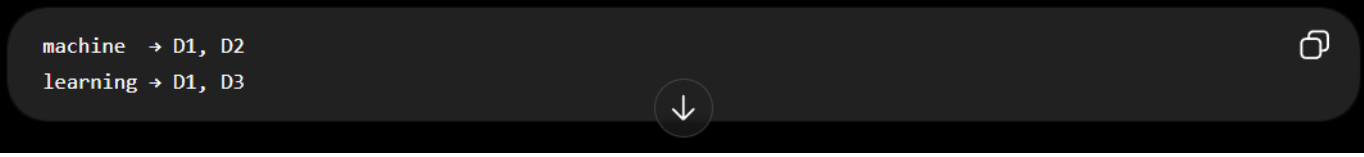
Documents are processed using techniques such as:

- Tokenization
- Stop-word removal
- Stemming
- Normalization

3. Index Construction

An **inverted index** is created by mapping terms to their corresponding documents.

Example:



4. Index Storage

The constructed index is stored in **memory or on disk**, depending on its size.

5. Query Processing

When a user submits a query, the index is searched to retrieve relevant documents.

6. Index Updating

When new documents are added or existing documents are modified/deleted, the index is updated.

For dynamic collections, a **temporary index** may be created and later merged with the main index.

7. Index Maintenance

The index is periodically maintained to:

- Merge indexes
- Remove obsolete entries
- Reorganize data
- Improve search performance

Example

Consider a search engine:

```
New Web Pages
     ↓
Process Documents
     ↓
Update Index
     ↓
User Searches
     ↓
Retrieve & Rank Results
     ↓
New Pages / Changes
     ↓
Update Index Again
```

Importance

The index life cycle helps to:

- Maintain **up-to-date search results**
- Improve retrieval speed
- Efficiently handle new and deleted documents
- Optimize storage
- Maintain index performance

Conclusion

The **Index Life Cycle** consists of **document collection, processing, index construction, storage, query processing, updating and maintenance**. It is a continuous process that keeps the IR system's index **accurate, efficient and up to date**.

16) What are the three components of query optimization?

Three Components of Query Optimization

Query Optimization is the process of selecting the most efficient way to process a query so that relevant results are obtained with minimum time and computational cost.

The **three main components** are:

1. Query Representation

The user's query is converted into a suitable internal representation that the IR system can process.

Example:

```
User Query
"machine learning"
      ↓
machine AND learning
```

It may involve tokenization, normalization, stop-word removal and query parsing.

2. Query Evaluation / Processing Strategy

The system decides **how the query should be executed efficiently**.

It may choose:

- DAAT or TAAT processing
- Order of query terms
- Appropriate posting lists
- Skip pointers
- Impact ordering

Example:

If one term has a very short posting list:

```
machine  → D1, D2
learning → D1, D2, D3, D4, D5, D6
```

the system can process **machine** first to reduce the number of candidate documents.

3. Cost Estimation and Selection

Different possible processing strategies are compared based on their **estimated cost**, such as:

- Number of postings accessed
- Number of comparisons
- Memory usage
- Disk I/O
- Processing time

The strategy with the **lowest estimated cost** is selected.

```
Possible Strategies
        ↓
Estimate Cost
        ↓
Compare Strategies
        ↓
Choose Cheapest Strategy
        ↓
Execute Query
```

In Short

Component	Purpose
Query Representation	Converts query into a processable form
Query Evaluation Strategy	Determines how the query will be processed
Cost Estimation & Selection	Selects the most efficient processing strategy

Thus, query optimization aims to reduce processing time and resource usage while producing the correct search results.

All the best

- **Karan Salunkhe** 😊